```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 5
Transmitted Code (with error induced at pos 5): 0110111
Received Code : 0110111
Checking for errors...
Error detected at bit position: 5 -> Correcting...
Corrected Code : 0110011
Recovered Data Bits : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 0
Transmitted Code (no error induced): 0110011
Received Code : 0110011
Checking for errors...
No error detected.
Corrected Code : 0110011
Recovered Data Bits : 1011
```

Exp.no: 6
Date: 8/8/26

Error Detection and Correction using Hamming code.

* Aim:

To write a program to implement error detection and correction using the Hamming code.

* Algorithm:

1. Start the program.
2. Read the number of 4-bit data blocks (n) forming the input data stream.
3. For each 4-bit data block $D = d_1 d_2 d_3 d_4$, perform Hamming code:

a) Arrange it bits with parity bits at position 1, 2 and 4 and data bits at position 3, 4, 6 and 7.

b) Compute the parity bits using even parity:

$P_1 = d_1 \text{ XOR } d_2 \text{ XOR } d_4.$

$P_2 = d_2 \text{ XOR } d_3 \text{ XOR } d_4.$

$P_4 = d_2 \text{ XOR } d_3 \text{ XOR } d_4.$

c) Form the 7 bit Hamming code word: $P_1 P_2 d_1 P_4 d_2 d_3 d_4$

4.) Simulate transmission by introduction of single bit error at a chosen bit position in code word.

5.) At the receiver, recompute the check bits from the received code word.

$C_1 = P_1 \text{ XOR } d_1 \text{ XOR } d_2 \text{ XOR } d_4$

$C_2 = P_2 \text{ XOR } d_1 \text{ XOR } d_3 \text{ XOR } d_4$

$C_4 = P_4 \text{ XOR } d_2 \text{ XOR } d_3 \text{ XOR } d_4$

6.) Compute Error position = $C_4 C_2 C_1$.

7.) If Error Position = 0 no error has occured.

Else, the bit at "Error Position" is inverted to correct the Error.

8.) Extract the original 4 data bits from the corrected 7 bit code word.

9.) Repeat steps 3 to 8 for all n blocks in the data stream

10.) Stop the program.

Program:

```
def encode(d):
    h = [0] * 7
    h[2], h[4], h[5], h[6] = d
    h[0], h[1], h[3] = h[2], h[4], h[6], h[2] ^ h[5]
                                ^ h[6], h[4], h[3] ^
                                h[6]
    return h

def detect_and_correct(h):
    c = (h[0] ^ h[2] ^ h[4] ^ h[6]) + 2 * (h[1] ^ h[2]
                                ^ h[5] ^ h[6]) +
                        4 * (h[3] ^ h[4] ^ h[5] ^ h[6])

if c:
    print(f"Error detected at bit position: {c}
                                -> correcting...")
    h[c-1] ^= 1
else:
    print("No error detected.")

def str(b):
    Return ''.join(map(str, b))

print("==> Hamming Code ...==\n")
for b in range(1, int(input("Enter number 4 bits:"))+1):
    print(f"\n--- Block {b} ---")
    d = list(map(int input("Enter 4 data bits (space-separated):
                                sp(,)[:4]))
```

```
h = encoder(d)
print(f"Original Data Bits : {''.join(map(str,d))}")
print(f"Encoded Hamming Code : {''.join(map(str,h))}")
p = int(input("Enter bit position : "))
if 1 <= p <= 7:
    h[p-1] ^= 1
    print(f"Transmitted code {p} : {''.join(map(str,h))}")
else:
    print(f"Transmitted Code (no Error): {''.join(map(str,h))}")
    print(f"Recieved Code : {''.join(map(str,h))}")
    print("Checking for error...")
    detect_and_correct(h)
    print(f"Corrected Code : {''.join(map(str,h))}")
    print(f"Recovered Data Bits : {''.join(map(str,[h[2],h[4],h[5],h[6]]))}")
```

* Result:

The program to implement error detection and correct using the Hamming code concept was successfully written in python, executed and verified using a sample input data stream.